

Подготовка к экзамену по C++ (ИТМО)

Тема: Лямбда-выражения (Под капотом)

Hardcore Theory Edition

19 июня 2026 г.

1 Анатомия Лямбды

Лямбда-выражение (появилось в C++11) — это синтаксический сахар для создания анонимных функций на лету. Синтаксис: `[захват](параметры) -> возвращаемый_тип { тело };`

Главный секрет: Лямбд не существует! Для компилятора C++ лямбда — это автоматически сгенерированный **безымянный класс** (Функтор), в котором перегружен `operator()`. Каждая написанная в коде лямбда имеет **свой уникальный, неповторимый тип**. Именно поэтому мы сохраняем их в `auto`.

2 Захват контекста (Captures)

Обычная функция видит только глобальные переменные и свои аргументы. Лямбда может "захватывать" переменные из функции, в которой она создана. Эти захваченные переменные становятся **приватными полями** сгенерированного класса!

2.1 Виды захвата

- `[]` — ничего не захватывать. (Такую лямбду можно скастовать к обычному Си-указателю на функцию!).
- `[=]` — захватить все используемые переменные **по значению** (сделать копию).
- `[&]` — захватить все используемые переменные **по ссылке** (сохранить указатель/ссылку в поле класса).
- `[x, &y]` — `x` скопировать, `y` взять по ссылке.

Суть на пальцах (Жизненная аналогия)

Лямбда — это шпион, которого вы отправляете на задание. `[=]` (по значению) — вы даете шпиону ксерокопии секретных документов. Если документы в штабе сгорят, у шпиона останутся копии. `[&]` (по ссылке) — вы даете шпиону рацию для связи со штабом. Если штаб уничтожат (функция завершится), шпион будет кричать в рацию в пустоту (Dangling Reference).

3 Проблема mutable

По умолчанию метод `operator()` у сгенерированного лямбда-класса помечен компилятором как `const`. Это значит, что если вы захватили переменную по значению `[x]`, вы **не можете** изменить её внутри лямбды (выкинет ошибку: *assignment of read-only variable*).

Чтобы разрешить лямбде менять свои **скопированные** поля, нужно добавить ключевое слово `mutable`:

```
1 int x = 0;
2 auto f = [x]() mutable {
3     x++; // Теперь это легально! Мы меняем КОПИЮ внутри лямбды.
4     std::cout << x;
5 };
```

4 Опасность: Данглинг (Dangling References)

Самый частый баг с лямбдами. Если лямбда захватывает локальную переменную по ссылке `[&]`, а затем эта лямбда возвращается из функции (или передается в другой поток), локальная переменная умирает (stack unwinding). Лямбда начинает указывать на уничтоженную память. **UB (Undefined Behavior)!**

5 Generic Лямбды (C++14)

Начиная с C++14, в параметрах лямбды можно писать `auto`.

```
1 auto print = [](auto x) { std::cout << x; };
```

Под капотом компилятор делает метод `operator()` **шаблонным** (`template <typename T> void operator()(T x)`). Это позволяет передавать в одну и ту же лямбду и `int`, и `std::string`.